

Shahoshi

Multimodal Edge-ML Women's Safety Wearable

Complete Implementation Plan & Technical Guide

Alerting: WiFi + Brevo WhatsApp Business API

Estimated hardware cost: BDT 1,850 - 2,000 per unit

Asif Zaman

25-93713-1@student.aiub.com

1. Executive Summary

Shahoshi is a wrist-worn safety device that passively monitors movement, heart rate, and ambient sound in real time. All anomaly detection runs entirely on-device using a quantized TinyML model — no internet connection is required for detection itself. When the device confirms a genuine distress pattern, it sounds a local alarm and sends a WhatsApp alert containing the wearer's live GPS location to a designated emergency contact, using the ESP32's WiFi connection and the Brevo messaging API.

The system is built around a **two-of-three sensor consensus** principle: a single sensor firing alone (e.g. elevated heart rate from exercise) does not trigger an alert. At least two of the three modalities — abnormal movement, heart-rate spike, and sudden loud sound — must agree before an SOS is raised. This is designed to minimize false positives while preserving sensitivity to real distress events.

2. Bill of Materials

Estimated cost per unit, sourced locally in Bangladesh:

Component	Qty	Function	Est. Cost (Taka)
ESP32 DevKit V1	1	Main MCU, inference engine, WiFi	485
MPU-6050 (GY-521)	1	6-axis IMU - movement anomaly	150
MAX30100 breakout	1	Optical PPG - heart rate anomaly	245
Sound Sensor (LM393, analog)	1	Ambient audio amplitude - scream detection	190
NEO-6M GPS module	1	Real-time location for alerts	550-700
Passive buzzer	1	Local audible alarm	30
TP4056 charger + 500mAh LiPo	1 each	Field power (no onboard charger on DevKit)	150-200
Jumper wires (assorted)	1 pack	Wiring	50

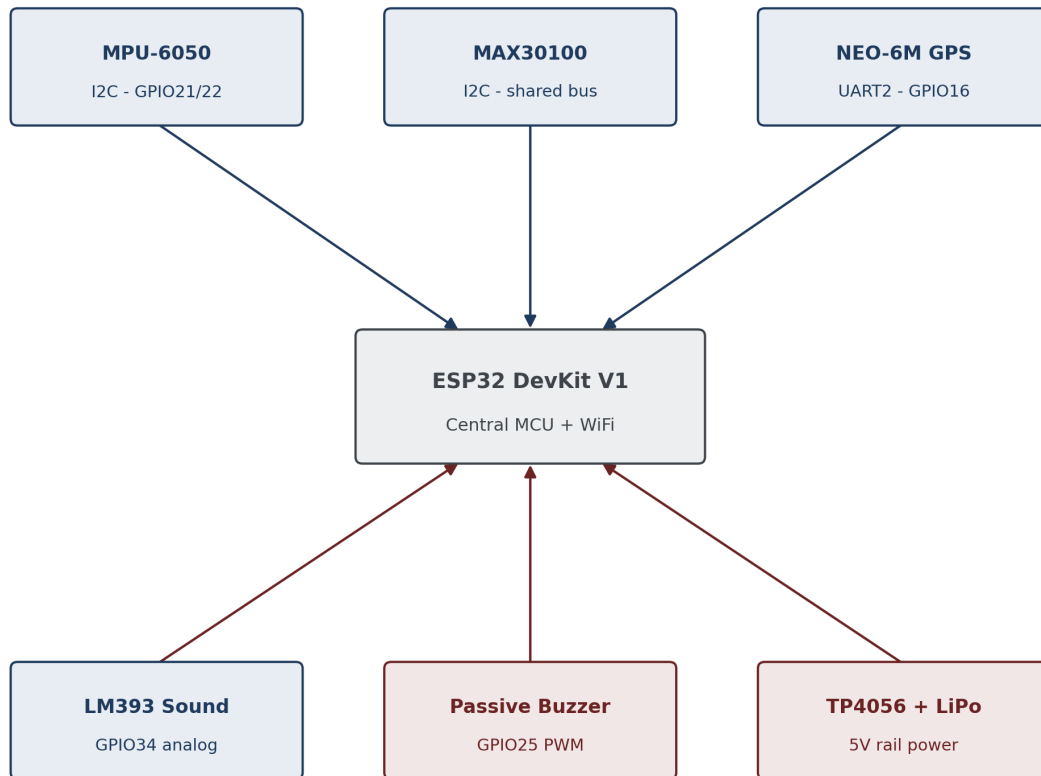
Total estimated cost: BDT 1,850 - 2,000 per unit

Note: an earlier design used a SIM800L GSM module for SMS alerts. This was replaced with the ESP32's built-in WiFi plus a cloud WhatsApp API (Brevo), which removes the SIM800L, its SIM card, and its power-buffering components (a common source of field failures), at the cost of requiring WiFi range and an active Brevo account with an approved message template.

3. Circuit & Wiring Diagram

All peripherals connect to the ESP32 over I2C (shared bus for the two motion/health sensors), a dedicated UART for GPS, and direct GPIO for the buzzer. Power is supplied through the TP4056/LiPo circuit feeding the ESP32's 5V input.

Shahoshi - Complete Wiring Diagram



Navy = sensor inputs Maroon = power / alert output Gray = controller

Figure 1. Complete wiring hub diagram — all components connect back to the ESP32 DevKit V1.

Component	Pin	ESP32 GPIO
MPU-6050	VCC / GND / SDA / SCL	3.3V / GND / GPIO21 / GPIO22
MAX30100	VIN / GND / SDA / SCL	3.3V / GND / GPIO21 / GPIO22 (shared I2C bus)
LM393 Sound Sensor	VCC / GND / A0	3.3V / GND / GPIO34
NEO-6M GPS	VCC / GND / TX	3.3V (or 5V per module) / GND / GPIO16 (RX2)
Passive Buzzer	+ / -	GPIO25 / GND
TP4056 + LiPo	OUT+ / OUT-	5V / GND (powers whole board)

Important: MPU-6050 and MAX30100 share the same I2C bus (GPIO21/22) but sit at different addresses (0x68 and 0x57 respectively), so no conflict occurs. The SIM800L 2A peak-current warning from earlier hardware iterations no longer applies since that module has been removed from the design.

4. System Architecture

The system has two distinct phases: an **offline training pipeline** that runs once on a PC to produce the anomaly-detection model, and a **real-time on-device pipeline** that runs continuously on the ESP32 using that trained model.

4.1 Offline Training Pipeline

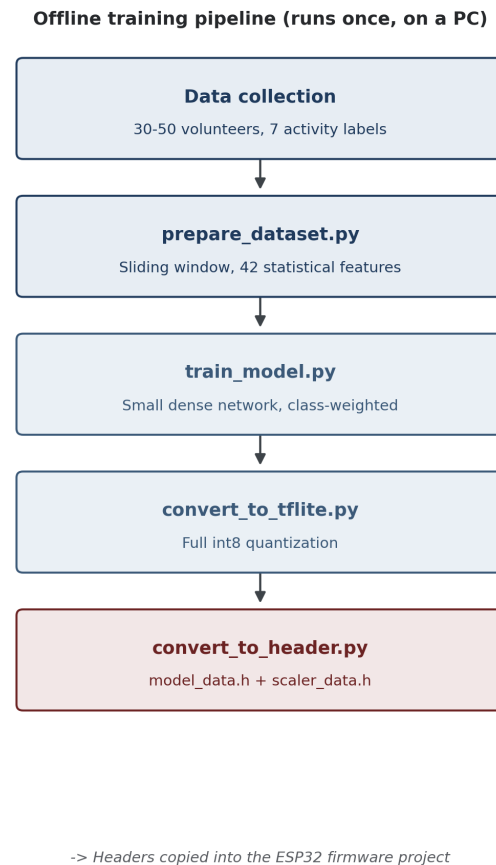


Figure 2. Data collected from volunteers is windowed into 42 statistical features, used to train a small quantization-friendly dense network, then converted to a C header for embedding in the firmware.

4.2 Real-Time On-Device Flow

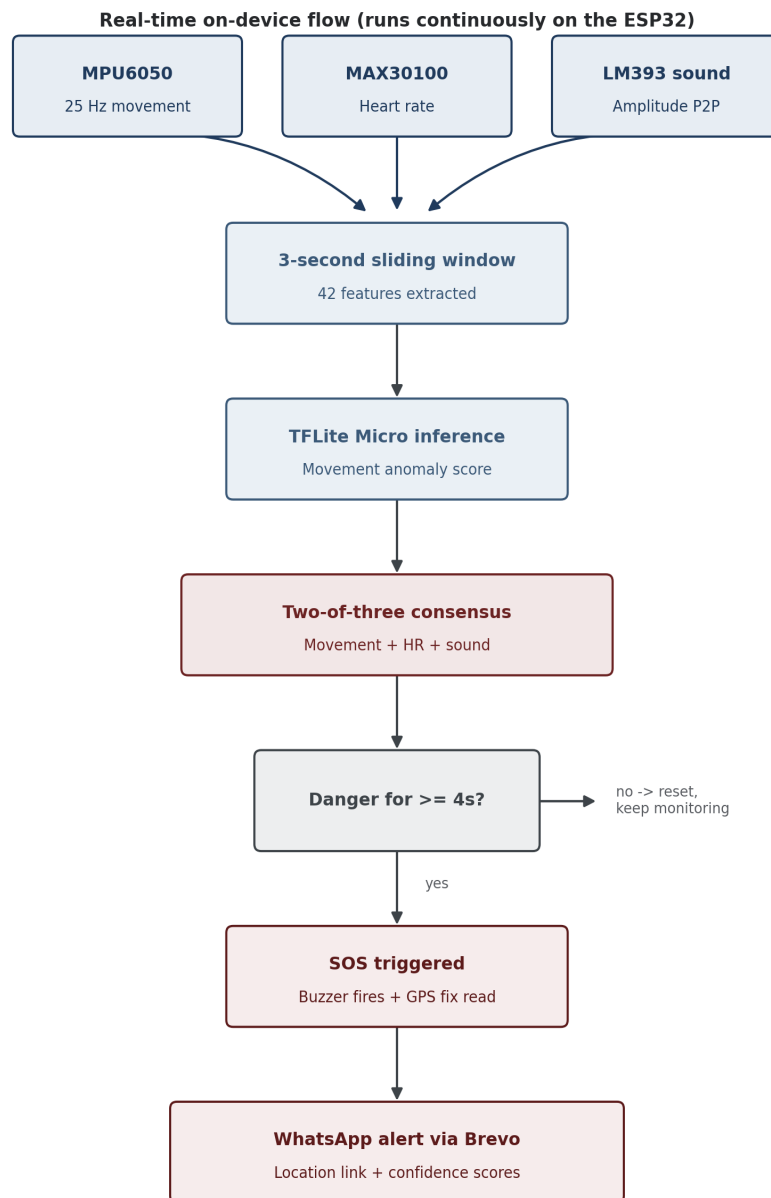


Figure 3. Continuous sensor sampling feeds a 3-second sliding window through the trained model; a two-of-three consensus sustained for 4 seconds triggers the buzzer and a WhatsApp alert with live GPS location.

5. Implementation Phases

Phase 1 - Sensor Validation

Before any hardware purchase is finalized, wiring and sensor logic are validated in the Wokwi circuit simulator. The ESP32, MPU-6050, and NEO-6M GPS are natively supported in simulation; MAX30100 and the LM393 sound sensor have no native simulator parts and are represented with potentiometer stand-ins for logic testing purposes only. A dedicated diagnostic firmware prints raw sensor values so each connection can be verified independently before integration.

- Pass criteria: MPU-6050 reads ~1.0g at rest and spikes when shaken; buzzer audibly pulses on boot; simulated HR and sound values sweep their expected ranges.
- Deliverables: diagram.json (simulator wiring), phase1_sensor_diagnostic.ino

Phase 2 - Data Collection & Preprocessing

Labeled sensor data is collected from 30-50 volunteers across seven activity classes (walking, standing, sitting, running, controlled struggle simulation, controlled fall simulation, and panic movement), following a supervised, consent-based protocol. Public datasets (UCI HAR for movement, WESAD for physiological stress) are used to bootstrap and sanity-check feature engineering, but the deployed model is trained primarily on data captured through the project's own sensor hardware, since public datasets use different sensor placements and hardware profiles.

- Raw sensor streams are windowed (3-second windows, 1-second step) and converted into 42 statistical features per window: mean, std, min, max, range, and RMS energy per IMU axis, plus signal magnitude area, heart-rate statistics, and sound amplitude statistics.
- Deliverables: prepare_dataset.py, labeled CSV dataset, processed train/val/test splits

Phase 3 - Model Training & Deployment

Model training happens entirely on a PC, not on the embedded device. A small dense neural network (64-32-16-1 units) is trained on the extracted features with class-weighting to address the natural imbalance between safe and danger examples, then converted to a fully int8-quantized TensorFlow Lite model (target size under 50KB) for deployment. The trained model and its feature-scaling parameters are then embedded into the firmware as C header files and flashed onto the ESP32 — the device performs inference locally without needing to contact the training pipeline again.

- Deliverables: train_model.py, convert_to_tflite.py, convert_to_header.py, model_data.h, scaler_data.h

Phase 4 - Testing & Validation

Once deployed, the system is validated against the performance targets in Section 6, with particular attention to false-positive rate across everyday activities (workout, commute, cooking) and confirmed true-positive detection of simulated distress scenarios. Threshold values are tuned iteratively based on field-testing results.

6. Testing & Validation Criteria

Metric	Minimum Acceptable	Target
Danger recall (true positive rate)	> 90%	> 95%
False positive rate	< 10%	< 2%
Inference latency	< 500 ms	< 200 ms
Battery life (LiPo, continuous monitoring)	> 6 hours	> 10 hours
Alert delivery time after confirmed trigger	< 15 seconds	< 8 seconds

Sensor-level pass criteria (verified in Phase 1 before integration):

Sensor	Pass Criteria
MPU6050	AZ - 1.0g at rest; spikes >2g when shaken
MAX30100	60-90 BPM at rest; rises to 100+ after exercise
LM393 Sound	P2P < 200 in a quiet room; > 800 when clapping nearby
NEO-6M GPS	Valid lat/long fix within 30-60 seconds outdoors

7. Communication & Alerting

On a confirmed danger event, the device performs two actions in parallel: sounding the local buzzer immediately, and sending a WhatsApp message to a pre-configured emergency contact via the Brevo transactional messaging API over WiFi.

7.1 Setup Requirements

- A Brevo account with the WhatsApp feature activated (requires linking a Facebook Business Manager and WhatsApp Business Account through Meta).
- An approved message **template** — WhatsApp does not allow free-form business-initiated messages; the first message in any conversation must use a template pre-approved by Meta, typically within a few hours to a couple of days.
- A Brevo API key, configured in the firmware alongside WiFi credentials and the emergency contact number (E.164 format).

7.2 Message Content

The alert template is filled with a Google Maps link built from the live GPS fix (or a fallback message if no fix is available), along with the confidence percentages from each of the three sensor modalities, so the recipient has both location and context immediately.

7.3 Known Limitation

This alerting path depends on WiFi availability at the time of the event. Unlike a cellular SMS/GSM approach, it will not function in areas without WiFi coverage. This trade-off was accepted in exchange for removing the SIM800L module's cost, wiring complexity, and known power-supply failure mode. If cellular-independent alerting becomes a requirement, an all-in-one GSM+GPS module (e.g. A9G) is the recommended alternative path, in place of the current WiFi + Brevo approach.

8. Software & Tooling Stack

Layer	Tooling
Firmware development	Arduino IDE 2.x or VS Code + PlatformIO
Circuit simulation	Wokwi (browser or VS Code extension)
On-device ML runtime	TensorFlow Lite for Microcontrollers
Model training	Python 3, TensorFlow/Keras, scikit-learn, pandas
Firmware libraries	MPU6050 (Electronic Cats), MAX30100lib, TinyGPSPPlus, TensorFlowLite_ESP32
Alerting	ESP32 WiFi, HTTPClient/WiFiClientSecure, Brevo WhatsApp API

9. Ethics & Safety Considerations

- Institutional ethics approval is required before recruiting volunteers for data collection.

- Written informed consent must be obtained from all participants.
 - Data collection sessions are framed to participants as a stress-detection study, not an assault simulation.
 - All simulated distress scenarios (struggle, fall) are conducted in controlled, supervised, safe conditions only — no real danger is introduced.
 - Raw audio is processed on-device into amplitude statistics only; raw audio is never stored or transmitted.
 - Collected data is anonymized before any sharing or publication.
-

10. Summary & Next Steps

- Procure the components listed in Section 2, verifying the sound sensor exposes an analog A0 pin and the GPS module includes an antenna.
- Complete Phase 1 sensor validation on physical hardware using the diagnostic firmware.
- Secure ethics approval and begin Phase 2 volunteer data collection.
- Train and quantize the model (Phase 3), then flash the final integrated firmware.
- Conduct Phase 4 field testing against the performance targets in Section 6, tuning thresholds as needed.
- Submit the Brevo WhatsApp template for approval early, in parallel with hardware procurement, since approval turnaround is outside the team's control.

Full firmware source, the Wokwi simulation files, and the Python training pipeline scripts referenced in this plan are maintained as a companion code package to this document.